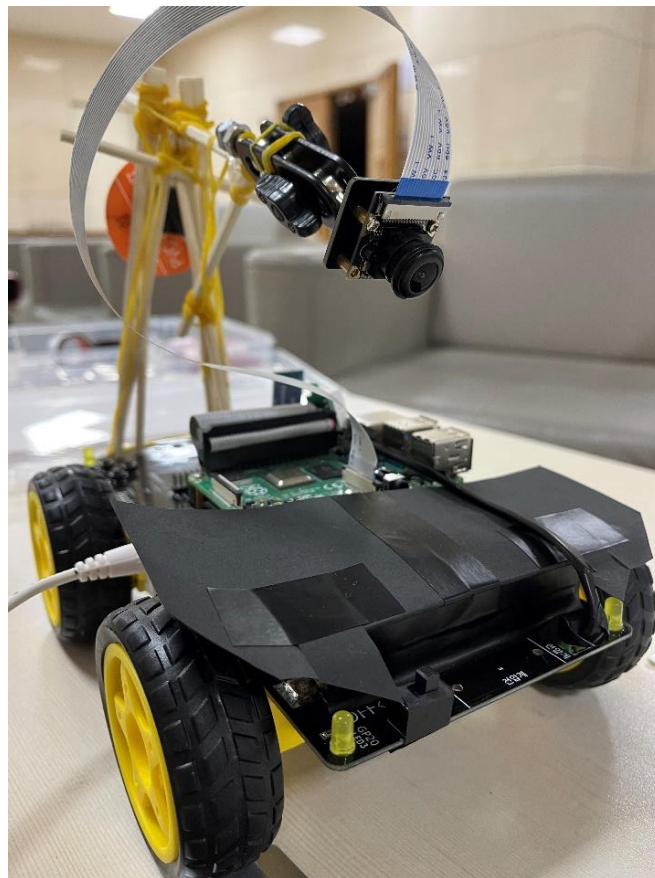


딥러닝 경로인식을 통한 자율주행 자동차제작

3조

2019204020	박우석
2017204035	강준석
2019204023	윤성호
2019204090	이재원



- 학습 데이터 취득 방법

광각 카메라를 통해 자동차 전방의 이미지를 1분에 160여장 수집함, 딥러닝 학습을 위해 수집되는 이미지는 사람이 직접 수동 조작을 하여 만들어진 트랙을 돌면서 이미지를 수집함. 수집된 이미지는 인식을 개선하기 위해 블러링 및 resize 등 전처리 과정을 거쳐 라즈베리파이에 저장된다. 차선의 인식율을 높이기 위하여 양쪽 차선이 보이는 부분만 크롭하여 라즈베리파이에 저장된다.

기존 카메라의 경우 거의 바로 앞의 지면만을 비추고 있었기에 보이는 라인이 상당히 적었다. 이에 대한 해결책으로 나무젓가락을 활용해 카메라의 각도를 수정했으며 상대적으로 튀어나온 길이가 짧은 뒷면을 앞면으로 이용해 회전시에 차량이 라인을 넘어가는 것을 최소화하였음.

도로의 상황에 따라 차선이 좁은 곳도, 넓은 곳도 있기에 여러 차선 넓이에서 학습을 진행함

여러 각도의 회전 도로를 만들어 학습을 진행함

-> 후진 주행을 위한 코드 수정(데이터 수집 코드, 주행 코드)

```
if keyValue == ord('q'):
    break
elif keyValue == 82:
    print("go")
    carState = "go"
    motor_back(speedSet)
elif keyValue == 84:
    print("stop")
    carState = "stop"
    motor_stop()
elif keyValue == 81:
    print("left")
    carState = "left"
    motor_left(speedSet)
elif keyValue == 83:
    print("right")
    carState = "right"
    motor_right(speedSet)

def motor_right(speed):
    L_Motor.ChangeDutyCycle(speed*SPEED.HIGH)
    GPIO.output(AIN2, False) # AIN2
    GPIO.output(AIN1, True) # AIN1
    R_Motor.ChangeDutyCycle(0)
    GPIO.output(BIN2, False) # BIN2
    GPIO.output(BIN1, True) # BIN1

def motor_left(speed):
    L_Motor.ChangeDutyCycle(0)
    GPIO.output(AIN2, False) # AIN2
    GPIO.output(AIN1, True) # AIN1
    R_Motor.ChangeDutyCycle(speed*SPEED.HIGH)
    GPIO.output(BIN2, False) # BIN2
    GPIO.output(BIN1, True) # BIN1
```

카메라의 위치를 변경하면서 기판의 앞뒤를 기존과 뒤집어 사용, 이에 따라 모터의 구동을 반대로 작동하도록 수정

-> 학습 데이터 사진 전처리 코드 수정

```
_, image = camera.read()
image = cv2.flip(image,-1)
height, _, _ = image.shape
image = image[int(height/2):,:,:]
image = cv2.cvtColor(image, cv2.COLOR_BGR2YUV)
image = cv2.resize(image, (200,66))
h,w = image.shape[:2]
image = image[:,int(w*0.1):] #Crop the image again to check
image = cv2.GaussianBlur(image,(5,5),0)
_,image = cv2.threshold(image,160,255,cv2.THRESH_BINARY_INV)
```

기존 코드는 기본 카메라 이미지의 너비 그대로, 높이는 절반 아랫 부분을 잘라서 이용했는데, 카메라 각도가 왼쪽 부분을 조금 더 많이 보여주는 이유로 너비 왼쪽 20 정도를 자르고 저장.

- 사용한 학습 모델 및 학습 방법

학습 모델 : nvidia 딥러닝 모델

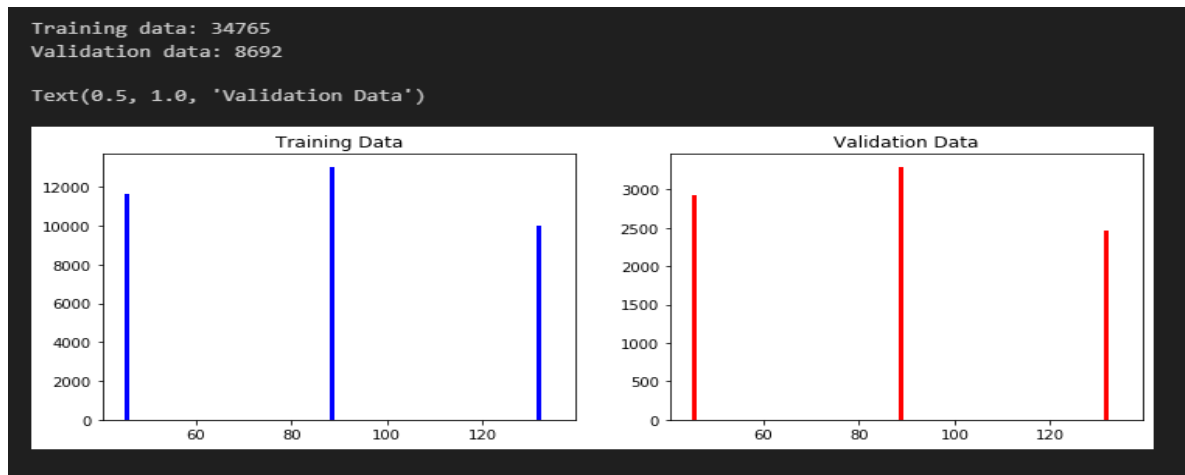
학습 방법 : 기본적으로 주어진 코드에서 몇 가지 파라미터를 수정해 가며 하이퍼 파라미터 튜닝 과정을 진행 후 가장 성능이 좋았던 모델을 이용

```
print(X_train, y_train)
history = model.fit_generator(image_data_generator(X_train, y_train, batch_size=100,
steps_per_epoch=350,
epochs=10,
validation_data = image_data_generator(X_valid, y_valid, batch_size=100),
validation_steps=200,
verbose=1,
shuffle=1,
callbacks=[checkpoint_callback])
```

시간 관계상 하이퍼 파라미터를 다양하게 적용해 보긴 힘들었지만, 같은 학습 데이터로 학습 시간이 너무 오래 걸리지 않는 선에서 다음과 같이 하이퍼 파라미터를 적용하였다.

```
r_squared = 82.15%
```

R-square 값은 처음 돌렸던 모델의 R-square 값 66퍼센트에서 더 많은 데이터 수집과 하이퍼 파라미터 튜닝으로 82퍼센트까지 올릴 수 있었고, 초기 학습시킨 모델보다 확연히 안정적이게 수행하였다.



> 최종적으로 수집한 데이터의 training-test 데이터 셋 분포

-제작한 자율주행 자동차 작동 영상

<https://photos.app.goo.gl/hDuKPRaTfHoiMQZU9>

- 학습 결과에 대한 discussion

데이터는 43000개 정도로 진행. R-square 값을 통해 성능 평가를 해보려 했으나 R-square 값이 높다고 무조건 주행을 잘 하는 것이 아니었음.

첫 번째 평가 주행 트랙처럼 간단한 맵에서는 속도 및 정확도 측면에서 성능이 좋은 편이었으나, 정면에 라인이 막혀 있는 경우 즉, 막 다른 길이나 급커브에서 라인을 따라 스스로 회전하지 못하고 그냥 직진 판단을 하는 경우가 실험 주행 중에 종종 있었음, 차량이 회전 중 카메라에 차선이 일시적으로 보이지 않으면 보이는 문제로 파악되어, 카메라의 위치 및 각도를 수정하여 해당 문제 발생 확률을 현저히 줄였으나 100% 해결하지는 못함, 실제 차량에는 카메라 및 라이다 센서 등 이미지를 수집하는 장치가 여러 개 있는 것으로 파악, 해당 실험에 사용된 차량에도 카메라가 여러 개였으면 자율주행의 정확도를 크게 증가시켰을 것으로 예상됨

추가적으로 평가주행 두 번째 골목길 맵에서 차선이 직각이고 넓은 트랙인 경우에 매우 낮은 정확도를 가졌음. 직각 넓은 차선에 대한 학습 데이터를 수집하지 않은 것을 원인으로 파악하고 있음

- 조원의 역할

강준석 : 학습 데이터 수집 코드 및 자율주행 주요 코드 튜닝

윤성호 : 딥러닝 모델 학습 및 하이퍼 파라미터 조정

이재원 : 데이터 학습 방법 개선, 주행동작 코드 수정

박우석 : 학습데이터 수집, 트랙제작

- 조원의 기여도

강준석(25%) + 윤성호(25%) + 이재원(25%) + 박우석(25%) = 100%